

FleetMan

Plateforme de Gestion de Flotte Intelligente

Intégration du Kernel RT-Comops

Analyse, Identification des Modules Applicables et Plan
d'Implémentation

Projet : **FleetMan — Module Fleet Management**

Kernel : **RT-Comops Backend — Architecture Multi-Tenant V3**

Type : **Document d'Intégration Technique**

Modules Kernel : **20 cores analysés, 9 applicables à FleetMan**

Version : **1.0 — Juin 2026**

Niveau : **5^{ème} année Génie Informatique**

Table des matières

Introduction Générale

Contexte du Projet

FleetMan est une plateforme de gestion de flotte de véhicules développée dans le cadre d'un projet de fin d'études de 5^{ème} année Génie Informatique. Elle constitue l'un des modules métier d'un écosystème applicatif plus vaste : la plateforme **RT-Comops**, un monolithe modulaire hexagonal multi-tenant développé par l'équipe de recherche et développement.

La plateforme RT-Comops est organisée autour d'un **Kernel** — le cœur technique transverse — qui concentre tous les mécanismes partagés entre les différents modules métier : authentification, gestion des identités, multi-tenancy, audit, événements, fichiers, organisations, rôles et permissions. Ce Kernel constitue la **base fondatrice** sur laquelle chaque module métier doit s'appuyer.

Objectif du Document

Ce document a pour objectif de :

1. **Analyser** en profondeur les 20 cores du Kernel RT-Comops et comprendre leur rôle fonctionnel et technique.
2. **Identifier** les modules du Kernel directement applicables et utiles au projet FleetMan.
3. **Expliquer** comment chaque module Kernel sélectionné sera utilisé dans FleetMan, avec les patterns d'intégration concrets.
4. **Planifier** l'implémentation de cette intégration de façon détaillée et progressive.

Architecture Générale du Kernel

Le Kernel RT-Comops est un monolithe modulaire hexagonal organisé en **20 cores fonctionnels**. Il repose sur la même stack technologique que FleetMan : Spring Boot, WebFlux, Java 21, R2DBC, PostgreSQL, Redis, Kafka. Cette cohérence technologique facilite considérablement l'intégration.

1. **kernel-core** — Multi-tenancy, JWT RS256, audit, outbox, ClientApplications
2. **common-core** — Types partagés, BaseEntity, Address, Contact, PartyRef
3. **actor-core** — Acteurs, BusinessActors, profils
4. **auth-core** — Authentification, OIDC, OTP, MFA, JWT
5. **roles-core** — Rôles, permissions, scopes RBAC
6. **organization-core** — Organisations, agences, points d'intérêt
7. **tp-core** — Tiers (clients, prospects, fournisseurs)
8. **settings-core** — Paramètres métier, séquences documentaires
9. **product-core** — Catalogue produits, variantes, prix
10. **inventory-core** — Stocks, mouvements, inventaires
11. **resource-core** — Ressources matérielles, véhicules, affectations
12. **sales-core** — Commandes de vente
13. **accounting-core** — Comptabilité générale
14. **billing-core** — Facturation commerciale
15. **treasury-core** — Trésorerie, comptes bancaires
16. **cashier-core** — Caisse opérationnelle
17. **administration-core** — Gouvernance, administration plateforme
18. **file-core** — Gestion des fichiers et documents
19. **hrm-core** — Ressources humaines
20. **blockchain-core** — Wallets, signatures, ancrage documentaire

Chapitre 1

Analyse Détaillée des 20 Cores du Kernel

1.1 kernel-core — La Colonne Vertébrale Technique

1.1.1 Description et Rôle Principal

Le `kernel-core` est le composant le plus fondamental de toute la plateforme. Il garantit l'exécution sûre de toutes les APIs en combinant :

- **Isolation multi-tenant logique** : chaque requête porte un `tenantId` injecté par le `TenantWebFilter` dans la chaîne réactive
- **Authentification des ClientApplications** : via les headers `X-Client-Id` et `X-API-Key`
- **Validation JWT RS256** : extraction des claims `sub`, `tid`, `oid`, `aid`, `actor`
- **Outbox transactionnel** : écriture atomique des événements dans `kernel.outbox_event`
- **Piste d'audit immuable** : `AuditTrail` en insertion seule
- **Quotas Redis** : limitation par `tenantId` + `clientId` + `serviceCode`

1.1.2 Contrat d'Appel Minimal

Tout backend qui se connecte au Kernel doit envoyer les headers suivants :

Listing 1.1 : Headers obligatoires vers le Kernel

```
X-Client-Id: <identifiant stable de la ClientApplication>
X-API-Key: <secret serveur de l'application>
X-Tenant-Id: <tenant_cible_de_l'appel>
Authorization: Bearer <accessToken> # requis pour actions utilisateur
X-Organization-Id: <requis pour services metier scopes organisation>
```

1.1.3 Catalogue des Services Autorisés

Le Kernel contrôle l'accès aux services via le catalogue `PlatformServiceCode` :

Code Service	Description
ORGANIZATION	Gestion des organisations et agences
SETTINGS	Paramètres métier et séquences
COMMERCIAL	Tiers, prospects, clients
PRODUCT	Catalogue produits
INVENTORY	Gestion des stocks
SALES	Commandes de vente
BILLING	Facturation commerciale
ACCOUNTING	Comptabilité générale
BANKING	Comptes bancaires
TREASURY	Trésorerie
CASHIER	Caisse opérationnelle
RESOURCE	Ressources matérielles
HRM	Ressources humaines
BLOCKCHAIN	Wallets et ancrage

TAB. 1.1 : Catalogue `PlatformServiceCode` du Kernel

1.2 common-core — Socle de Types Partagés

1.2.1 Description

Le `common-core` fournit les abstractions réutilisées par tous les autres cores :

- **BaseEntity** : classe abstraite avec UUID `id`, `tenantId`, `createdAt`, `updatedAt`
- **Address** : adresse polymorphe attachable à toute entité (lignes, ville, pays, GPS)
- **Contact** : contact polymorphe avec vérification email/téléphone
- **PartyRef** : référence légère inter-cores (`partyType` + `partyId`)
- **ApiResponse<T>** et **PageResult<T>** : enveloppes de réponse JSON uniformes
- **PlatformServiceCode** : énumération partagée des services

1.2.2 Invariants Clés

- Une seule adresse par défaut par combinaison (`addressableId`, `addressableType`, `addressType`)
- `PartyRef.partyType` limité à `ACTOR` et `ORGANIZATION`
- `tenantId` obligatoire sur toutes les structures polymorphiques

1.3 actor-core — Gestion des Acteurs

1.3.1 Description

L'`actor-core` gère les acteurs de la plateforme : personnes physiques (`Actor`), entités commerciales (`BusinessActor`), et leurs profils détaillés. Il couvre :

- Création, mise à jour et suppression douce des acteurs
- Rattachement d'un acteur à une organisation
- Cycle de vie : `PENDING` → `ACTIVE` → `SUSPENDED` → `DELETED`
- Réactivation par auto-service ou approbation administrative

1.4 auth-core — Authentification et Identité

1.4.1 Description

L'`auth-core` gère l'ensemble du cycle d'authentification :

- Création de comptes utilisateurs par un administrateur IAM
- Authentification locale tenant-scoped avec émission de JWT RS256
- Découverte globale des contextes de login (multi-tenant)
- Sélection explicite d'un contexte tenant/organisation
- Identification email-first et inscription contextuelle
- Réinitialisation de mot de passe (SMTP ou `PREVIEW_ONLY`)
- Vérification d'email post-authentification
- Gestion du profil utilisateur (`/users/me`)

1.4.2 Flow JWT RS256

Le Kernel émet des JWT RS256 signés avec une clé privée. Les backends consommateurs vérifient la signature via le document `/.well-known/jwks.json`. Les claims transportés sont : `sub` (userId), `tid` (tenantId), `oid` (organizationId), `aid` (agencyId), `actor` (actorId).

1.5 roles-core — Rôles et Permissions RBAC

1.5.1 Description

Le `roles-core` implémente un système RBAC (Role-Based Access Control) complet :

- Création de rôles avec codes de permission granulaires
- Affectation de rôles à des utilisateurs avec portée (TENANT, ORGANIZATION, AGENCY)
- Résolution des permissions avec cache Redis optionnel
- Construction des autorités scopées à partir d’une affectation
- Invalidation du cache après modification d’affectation
- Provisionnement de templates de rôles (réservé à `administration-core`)

1.6 organization-core — Organisations et Agences

1.6.1 Description

L’`organization-core` gère la structure organisationnelle :

- Création d’une organisation avec agence siège automatique
- Création d’agences opérationnelles
- Gestion des horaires d’ouverture
- Lien fonctionnel acteur-organisation
- Suspension d’une organisation et de ses agences
- Ajout de points d’intérêt géographiques (GPS)

1.7 tp-core — Gestion des Tiers

1.7.1 Description

Le **tp-core** gère les tiers commerciaux (clients, prospects, fournisseurs) :

- Création d'un tiers avec rôle client
- Qualification commerciale d'un prospect
- Conversion prospect → client
- Enregistrement d'interactions commerciales
- Ajout de comptes bancaires
- Recherche plein-texte (Elasticsearch / fallback PostgreSQL)

1.8 settings-core — Paramètres Métier

1.8.1 Description

Le **settings-core** gère la configuration métier de la plateforme :

- Génération de numéros documentaires avec fallback agence → organisation
- Lecture des paramètres métier effectifs
- Mise à jour des options générales d'une organisation
- Gestion des séquences documentaires par agence
- Politique opérationnelle d'agence avec repli organisation

1.9 resource-core — Ressources Matérielles

1.9.1 Description

Le **resource-core** est le module le plus directement lié à FleetMan. Il gère les ressources matérielles de l'organisation :

- Enregistrement d'une ressource matérielle (véhicule, équipement)
- Affectation d'une ressource à un acteur (conducteur)
- Envoi en maintenance et retour à l'état opérationnel

- Mise à jour de la localisation GPS en temps réel
- Mise au rebut irréversible

1.10 file-core — Gestion des Fichiers

1.10.1 Description

Le `file-core` gère le cycle de vie des fichiers et documents :

- Téléversement avec audit système
- Téléchargement sécurisé
- Rattachement d'un document à une cible métier (polymorphique)
- Consultation des documents d'une cible
- Politique de gouvernance documentaire (expiration, révision)

1.11 accounting-core, billing-core, treasury-core — Finance

Ces trois cores couvrent le cycle financier complet :

- **accounting-core** : Comptabilité générale, factures canoniques, règlements, postings
- **billing-core** : Facturation commerciale legacy, bons de livraison, paiements
- **treasury-core** : Comptes bancaires, relevés, rapprochements, chèques

1.12 hrm-core — Ressources Humaines

Le `hrm-core` couvre l'ensemble du cycle RH : employés, contrats, congés, paie, recrutement, onboarding, formation, frais, missions, médical et KPI RH. Il s'intègre avec `actor-core` et `organization-core`.

1.13 blockchain-core — Ancrage et Wallets

Le `blockchain-core` fournit une blockchain interne pour : wallets, signatures numériques, blocs, ancrage documentaire et ancrage d'événements métier. Il garantit l'immuabilité et la traçabilité des opérations critiques.

1.14 administration-core — Gouvernance

L'**administration-core** gère la gouvernance de la plateforme : approbation des BusinessActors, mise à jour des options de plateforme avec audit, consultation de la piste d'audit, supervision de l'état du système.

Chapitre 2

Identification des Modules Kernel Applicables à FleetMan

2.1 Méthodologie de Sélection

Pour identifier les modules Kernel applicables à FleetMan, nous avons appliqué trois critères de sélection :

1. **Pertinence fonctionnelle** : le module couvre un besoin déjà présent ou planifié dans FleetMan
2. **Évitement de la duplication** : FleetMan ne doit pas réimplémenter ce que le Kernel fournit déjà
3. **Cohérence architecturale** : l'intégration respecte le contrat backend-to-kernel

2.2 Tableau de Sélection des Modules

Module Kernel	Applicable	Justification
kernel-core	OUI	Fondation obligatoire : multi-tenancy, JWT, audit, outbox
common-core	OUI	BaseEntity, Address, Contact, PageResult réutilisables
actor-core	OUI	Gestionnaires et conducteurs sont des acteurs
auth-core	OUI	Remplace l'authentification locale actuelle de FleetMan
roles-core	OUI	Remplace le RBAC local (FLEET_MANAGER, FLEET_DRIVER)

Module Kernel	Applicable	Justification
organization-core	OUI	Une flotte = une organisation, un dépôt = une agence
tp-core	OUI	Les clients des missions sont des tiers
settings-core	OUI	Numéros de missions, paramètres métier
resource-core	OUI	Les véhicules sont des ressources matérielles
file-core	OUI	Documents légaux véhicules/conducteurs
billing-core	PARTIEL	Facturation des missions (Module 7 du cahier)
accounting-core	PARTIEL	Comptabilisation des dépenses opérationnelles
product-core	NON	Pas de catalogue produits dans FleetMan
inventory-core	NON	Pas de gestion de stocks dans FleetMan
sales-core	NON	Pas de commandes de vente au sens strict
treasury-core	NON	Hors périmètre FleetMan
cashier-core	NON	Hors périmètre FleetMan
administration-core	PARTIEL	Gouvernance du super-admin FleetMan
hrm-core	NON	Hors périmètre FleetMan (RH complète)
blockchain-core	FUTUR	Ancrage des incidents critiques (phase future)

TAB. 2.1 : Sélection des modules Kernel applicables à FleetMan

2.3 Les 9 Modules Retenus

Modules Kernel retenus pour FleetMan

1. **kernel-core** — Fondation obligatoire (multi-tenancy, JWT, audit, out-box)
2. **common-core** — Types partagés (BaseEntity, Address, Contact, Page-Result)
3. **actor-core** — Gestion des acteurs (gestionnaires, conducteurs)
4. **auth-core** — Authentification centralisée (remplace l'auth locale)
5. **roles-core** — RBAC centralisé (remplace les rôles locaux)
6. **organization-core** — Structure organisationnelle (flotte = organisation)
7. **tp-core** — Gestion des tiers (clients des missions)
8. **settings-core** — Paramètres métier et numérotation
9. **resource-core** — Ressources matérielles (véhicules)
10. **file-core** — Documents légaux (assurance, permis, carte grise)

Chapitre 3

Analyse Détaillée de Chaque Intégration

3.1 Intégration 1 — kernel-core : Fondation Multi-Tenant

3.1.1 Situation Actuelle dans FleetMan

Actuellement, FleetMan gère son propre contexte de sécurité via `SecurityConfig`, `JwtAuthenticationManager` et `BearerTokenServerAuthenticationConverter`. L'authentification est déléguée à un service Auth externe via `RemoteAuthAdapter`, mais sans le contrat formel du Kernel.

3.1.2 Ce que le kernel-core Apporte

- **Multi-tenancy logique** : FleetMan devient un tenant du Kernel. Chaque entreprise de transport est un tenant distinct, avec isolation complète des données.
- **ClientApplication** : FleetMan s'enregistre comme `fleet-management-backend` avec ses `allowedServiceCodes` : `ORGANIZATION`, `RESOURCE`, `COMMERCIAL`, `SETTINGS`, `FILE`.
- **JWT RS256 centralisé** : FleetMan vérifie les tokens via le JWKS du Kernel au lieu de son propre mécanisme.
- **Audit immuable** : toutes les opérations critiques (création véhicule, incident, maintenance) sont tracées dans `kernel.outbox_event`.
- **Outbox transactionnel** : les événements FleetMan (incident critique, maintenance créée) sont publiés via l'outbox du Kernel vers Kafka.

3.1.3 Mapping Conceptuel

Concept FleetMan	Concept Kernel	Mapping
Entreprise de transport	Tenant	1 :1
Application FleetMan	ClientApplication	1 :1
Token JWT actuel	JWT RS256 Kernel	Remplacement
Audit local	AuditTrail Kernel	Remplacement
Événements Kafka locaux	Outbox Kernel	Enrichissement

TAB. 3.1 : Mapping kernel-core vers FleetMan

3.1.4 Implémentation Technique

Listing 3.1 : Configuration de la ClientApplication FleetMan

```

1 // Dans application.yml
2 application:
3   kernel:
4     url: https://kernel.rt-comops.com
5     client-id: fleet-management-backend
6     api-key: ${FLEET_KERNEL_API_KEY}
7     tenant-id: ${FLEET_TENANT_ID}
8     allowed-services:
9       - ORGANIZATION
10      - RESOURCE
11      - COMMERCIAL
12      - SETTINGS
13      - FILE

```

Listing 3.2 : Adapter WebClient vers le Kernel

```

1 @Component
2 @RequiredArgsConstructor
3 public class KernelWebClientAdapter {
4
5     private final WebClient kernelWebClient;
6
7     @Value("${application.kernel.client-id}")
8     private String clientId;
9
10    @Value("${application.kernel.api-key}")
11    private String apiKey;
12
13    @Value("${application.kernel.tenant-id}")
14    private String tenantId;

```



```

15
16 public WebClient.RequestHeadersSpec<?> withKernelHeaders(
17     WebClient.RequestHeadersSpec<?> spec,
18     String organizationId,
19     String bearerToken) {
20     return spec
21         .header("X-Client-Id", clientId)
22         .header("X-API-Key", apiKey)
23         .header("X-Tenant-Id", tenantId)
24         .header("X-Organization-Id", organizationId)
25         .header("Authorization", "Bearer " + bearerToken);
26     }
27 }

```

3.2 Intégration 2 — common-core : Types Partagés

3.2.1 Situation Actuelle

FleetMan utilise ses propres records Java immuables (`Vehicle`, `Driver`, `Fleet`) sans base commune. La pagination est gérée par le `PageResponse<T>` créé dans le Chapitre 1 de notre implémentation.

3.2.2 Ce que le common-core Apporte

- **BaseEntity** : FleetMan adopte la classe abstraite `BaseEntity` du Kernel comme base de toutes ses entités, garantissant la présence de `tenantId`, `createdAt`, `updatedAt`.
- **Address polymorphique** : les adresses des flottes, dépôts et conducteurs utilisent le modèle `Address` du Kernel au lieu de champs texte dispersés.
- **Contact polymorphique** : les contacts des gestionnaires et conducteurs utilisent `Contact` du Kernel.
- **PageResult<T>** : notre `PageResponse<T>` est aligné sur le format `PageResult<T>` du Kernel pour une cohérence API globale.
- **ApiResponse<T>** : toutes les réponses FleetMan adoptent l'enveloppe standard du Kernel.

3.2.3 Mapping Conceptuel

Concept FleetMan	Concept Kernel	Action
Champs <code>createdAt/updatedAt</code>	<code>BaseEntity</code>	Héritage
Adresse de la flotte	<code>Address</code> polymorphique	Remplacement
Contact du gestionnaire	<code>Contact</code> polymorphique	Remplacement
<code>PageResponse<T></code>	<code>PageResult<T></code>	Alignement
Réponses REST	<code>ApiResponse<T></code>	Adoption

TAB. 3.2 : Mapping common-core vers FleetMan

3.3 Intégration 3 — auth-core : Authentification Centralisée

3.3.1 Situation Actuelle

FleetMan délègue déjà l'authentification à un service Auth externe via `RemoteAuthAdapter` et `AuthApiClient`. Le `JwtAuthenticationManager` valide les tokens en interrogeant ce service.

3.3.2 Ce que l'auth-core Apporte

L'auth-core du Kernel **remplace complètement** le service Auth externe actuel. Les bénéfices sont :

- **Login multi-contexte** : un utilisateur peut avoir des comptes dans plusieurs tenants (plusieurs entreprises de transport)
- **Découverte des contextes** : l'endpoint `/auth/contexts` liste les tenants disponibles pour un email donné
- **JWT RS256 standardisé** : les claims `tid`, `oid`, `aid` sont automatiquement présents
- **OTP et MFA** : sécurité renforcée pour les gestionnaires de flotte
- **Réinitialisation de mot de passe** : flux complet avec SMTP
- **Vérification d'email** : obligatoire pour les nouveaux comptes

3.3.3 Mapping des Rôles

Rôle FleetMan Actuel	Rôle Kernel	Scope
SUPER_ADMIN	SYSTEM_ADMIN	SYSTEM
FLEET_ADMIN	TENANT_ADMIN	TENANT
FLEET_MANAGER	ORG_MANAGER	ORGANIZATION
FLEET_DRIVER	ORG_OPERATOR	ORGANIZATION

TAB. 3.3 : Mapping des rôles FleetMan vers le Kernel

3.3.4 Adaptation du SecurityConfig

Listing 3.3 : Nouveau SecurityConfig avec JWT Kernel RS256

```

1 @Configuration
2 @EnableWebFluxSecurity
3 @EnableReactiveMethodSecurity
4 public class SecurityConfig {
5
6     @Bean
7     public SecurityWebFilterChain filterChain(ServerHttpSecurity http,
8         KernelJwtAuthenticationManager authManager) {
9         return http
10             .cors(cors -> cors.configurationSource(corsSource()))
11             .csrf(ServerHttpSecurity.CsrfSpec::disable)
12             .authorizeExchange(ex -> ex
13                 .pathMatchers("/v3/api-docs/**", "/swagger-ui/**",
14                     "/actuator/**", "/api/v1/auth/**").
15                     permitAll()
16                 .anyExchange().authenticated()
17             )
18             // Validation JWT via JWKS du Kernel
19             .oauth2ResourceServer(oauth2 -> oauth2
20                 .jwt(jwt -> jwt.authenticationManager(authManager))
21             )
22             .build();
23 }

```

3.4 Intégration 4 — roles-core : RBAC Centralisé

3.4.1 Situation Actuelle

FleetMan utilise `@PreAuthorize("hasRole('FLEET_MANAGER')")` avec des rôles définis localement dans le JWT. Le `JwtAuthenticationManager` extrait les rôles du token.

3.4.2 Ce que le roles-core Apporte

- **Rôles avec portée** : un rôle peut être limité à une organisation ou une agence spécifique
- **Permissions granulaires** : au lieu de rôles grossiers, des codes de permission précis (VEHICLE_CREATE, DRIVER_MANAGE, INCIDENT_REPORT)
- **Cache Redis** : les permissions sont mises en cache pour des performances optimales
- **Templates de rôles** : des rôles prédéfinis pour les cas d'usage FleetMan

3.4.3 Définition des Rôles FleetMan dans le Kernel

Listing 3.4 : Templates de rôles FleetMan à provisionner dans le Kernel

```
roles:
- code: FLEET_MANAGER
  name: "Gestionnaire de Flotte"
  permissions:
    - VEHICLE_CREATE
    - VEHICLE_READ
    - VEHICLE_UPDATE
    - VEHICLE_DELETE
    - DRIVER_CREATE
    - DRIVER_READ
    - DRIVER_MANAGE
    - FLEET_MANAGE
    - INCIDENT_READ
    - MAINTENANCE_MANAGE
    - ASSIGNMENT_MANAGE
    - DOCUMENT_MANAGE
    - KPI_READ
    - EXPENSE_APPROVE

- code: FLEET_DRIVER
  name: "Conducteur"
  permissions:
    - VEHICLE_READ
    - TRIP_START
    - TRIP_END
    - TELEMETRY_SEND
    - INCIDENT_REPORT
    - MAINTENANCE_DECLARE
    - FUEL_RECHARGE_CREATE
    - ASSIGNMENT_VIEW
    - DOCUMENT_VIEW
```

3.5 Intégration 5 — organization-core : Flotte = Organisation

3.5.1 Mapping Conceptuel Clé

C'est l'une des intégrations les plus structurantes. Le mapping est le suivant :

Concept FleetMan	Concept Kernel	Cardinalité
Entreprise de transport	Tenant	1 :1
Flotte de véhicules	Organisation	1 :1
Dépôt / Garage	Agence	N :1
Gestionnaire de flotte	Acteur lié à l'organisation	N :1
Zone géographique	Point d'intérêt d'agence	N :1

TAB. 3.4 : Mapping organization-core vers FleetMan

3.5.2 Conséquences Architecturales

- La table `fleet.fleets` devient une **projection locale** de l'organisation Kernel, enrichie des données spécifiques à FleetMan (nombre de véhicules, etc.)
- L'`organizationId` du Kernel est propagé dans tous les appels FleetMan via le header `X-Organization-Id`
- La création d'une flotte dans FleetMan déclenche la création d'une organisation dans le Kernel (via l'outbox)
- La suspension d'une organisation dans le Kernel suspend automatiquement la flotte correspondante

3.5.3 Synchronisation via Outbox

Listing 3.5 : Synchronisation flotte-organisation via événements Kafka

```

1 // Consommateur Kafka dans FleetMan
2 @Component
3 @RequiredArgsConstructor
4 @Slf4j
5 public class OrganizationEventConsumer {

```

```

6
7     private final FleetRepositoryPort fleetPort;
8
9     @KafkaListener(topics = "iwm.events.business",
10                    groupId = "fleet-management-group")
11     public void onOrganizationEvent(OrganizationEvent event) {
12         if ("ORGANIZATION_SUSPENDED".equals(event.type())) {
13             // Suspendre la flotte correspondante
14             fleetPort.findByOrganizationId(event.organizationId())
15                 .flatMap(fleet -> fleetPort.suspend(fleet.id()))
16                 .subscribe();
17         }
18     }
19 }

```

3.6 Intégration 6 — tp-core : Clients des Missions

3.6.1 Situation Actuelle

Le Module 7 (Clients et Missions) de notre cahier d'analyse prévoit une table `fleet.clients` locale. Avec le Kernel, ces clients deviennent des **tiers** gérés par **tp-core**.

3.6.2 Ce que le tp-core Apporte

- **Cycle de vie complet** : prospect → client avec qualification commerciale
- **Recherche plein-texte** : via Elasticsearch pour retrouver rapidement un client
- **Comptes bancaires** : pour la facturation des missions
- **Historique des interactions** : toutes les missions sont des interactions commerciales
- **Déduplication** : un client peut être partagé entre plusieurs flottes du même tenant

3.6.3 Adaptation du Module 7

La table `fleet.clients` locale est remplacée par une **référence légère** vers le tiers Kernel :

Listing 3.6 : Adaptation de la table missions avec référence tiers Kernel

```

1 -- Adaptation de fleet.missions

```

```
2 ALTER TABLE fleet.missions
3     ADD COLUMN IF NOT EXISTS third_party_id UUID, -- ID du tiers dans
        le Kernel
4     ADD COLUMN IF NOT EXISTS third_party_name VARCHAR(255); --
        Dénormalisé
5 -- La colonne client_id locale devient optionnelle (migration
        progressive)
```

3.7 Intégration 7 — settings-core : Paramètres et Numérotation

3.7.1 Ce que le settings-core Apporte à FleetMan

- **Numéros de missions** : génération automatique de références uniques (MSN-2026-001234) avec fallback agence → organisation
- **Numéros de factures** : séquences documentaires pour les factures de missions
- **Paramètres métier** : taux de TVA, devise par défaut (XAF), limites opérationnelles
- **Politique opérationnelle** : horaires de service, zones autorisées par agence

3.7.2 Utilisation dans FleetMan

Listing 3.7 : Appel au settings-core pour générer un numéro de mission

```
1 @Component
2 @RequiredArgsConstructor
3 public class MissionNumberGenerator {
4
5     private final KernelWebClientAdapter kernelClient;
6
7     public Mono<String> generateMissionNumber(String organizationId,
8                                             String agencyId,
9                                             String bearerToken) {
10         return kernelClient.withKernelHeaders(
11             kernelWebClient.post()
12                 .uri("/api/v1/settings/sequences/next")
13                 .bodyValue(Map.of(
14                     "sequenceCode", "MISSION",
15                     "agencyId", agencyId
16                 )),
17             organizationId, bearerToken
```

```

18         ).retrieve()
19         .bodyToMono(String.class);
20     }
21 }

```

3.8 Intégration 8 — resource-core : Véhicules comme Ressources

3.8.1 Situation Actuelle

FleetMan gère ses véhicules localement via `VehicleService` et `VehiclePersistenceAdapter`. Il existe déjà un `ExternalVehiclePort` et un `VehicleApiAdapter` qui communiquent avec un service véhicule externe.

3.8.2 Ce que le resource-core Apporte

Le **resource-core** du Kernel **remplace** le service véhicule externe actuel. Les véhicules deviennent des **ressources matérielles** du Kernel :

- **Enregistrement centralisé** : un véhicule est enregistré une fois dans le Kernel et référencé dans FleetMan
- **Affectation standardisée** : l'affectation conducteur-véhicule utilise le mécanisme d'affectation acteur-ressource du Kernel
- **Cycle de vie unifié** : AVAILABLE → ON_TRIP → MAINTENANCE → DECOMMISSIONED
- **Localisation GPS** : mise à jour de la position via le port dédié du Kernel
- **Maintenance intégrée** : le passage en maintenance est synchronisé avec le Kernel

3.8.3 Mapping Technique

Concept FleetMan	Concept Kernel	Action
Vehicle	Resource (type VEHICLE)	Synchronisation
VehicleService.create()	resource-core.register()	Délégation
VehicleService.assignDriver()	resource-core.assign()	Délégation
TripService.startTrip()	resource-core.updateGps()	Enrichissement

Concept FleetMan	Concept Kernel	Action
MaintenanceService.createVehicle()	resource-core.sendToMaintenance()	Registration

TAB. 3.5 : Mapping resource-core vers FleetMan

3.8.4 Adaptation du VehicleService

Listing 3.8 : VehicleService enrichi avec resource-core

```
1 @Service
2 @RequiredArgsConstructor
3 @Slf4j
4 public class VehicleService implements ManageVehicleUseCase {
5
6     private final VehiclePersistencePort vehiclePort;
7     private final KernelResourceAdapter kernelResourceAdapter;
8
9     @Override
10    public Mono<Vehicle> createIndependentVehicle(VehicleRequest
11        request,
12                                                UUID managerId,
13                                                String token) {
14        // 1. Enregistrer dans le Kernel resource-core
15        return kernelResourceAdapter.registerResource(
16            ResourceRegistrationRequest.builder()
17                .resourceType("VEHICLE")
18                .name(request.brand() + " " + request.model())
19                .serialNumber(request.vehicleSerialNumber())
20                .organizationId(managerId.toString())
21                .build(), token
22        )
23        // 2. Sauvegarder localement avec l'ID Kernel
24        .flatMap(kernelResource -> {
25            Vehicle vehicle = buildVehicle(request, managerId,
26                kernelResource.id());
27            return vehiclePort.saveLocalData(vehicle);
28        });
29    }
```

3.9 Intégration 9 — file-core : Documents Légaux

3.9.1 Situation Actuelle

Le Module 2 (Documents Légaux) de notre implémentation stocke les URLs des fichiers dans les tables `fleet.vehicle_documents` et `fleet.driver_documents`. Les fichiers eux-mêmes sont stockés localement ou sur un service externe non standardisé.

3.9.2 Ce que le file-core Apporte

- **Stockage centralisé** : tous les fichiers (assurance, permis, carte grise) sont gérés par le Kernel
- **Rattachement polymorphique** : un document est attaché à une cible métier (véhicule ou conducteur) via `targetType` + `targetId`
- **Politique de gouvernance** : expiration automatique, révision périodique
- **Audit de téléversement** : chaque upload est tracé dans l'audit Kernel
- **Téléchargement sécurisé** : accès contrôlé par les permissions RBAC

3.9.3 Adaptation du DocumentService

Listing 3.9 : DocumentService enrichi avec file-core

```

1  @Override
2  public Mono<VehicleDocument> addVehicleDocument(
3      AddVehicleDocumentCommand cmd) {
4
5      // 1. Téléverser le fichier dans le Kernel file-core
6      return kernelFileAdapter.uploadFile(
7          cmd.fileContent(), cmd.fileName(),
8          "VEHICLE_DOCUMENT", cmd.vehicleId().toString()
9      )
10     // 2. Récupérer l'URL sécurisée du Kernel
11     .flatMap(kernelFile -> {
12         VehicleDocument doc = new VehicleDocument(
13             null, cmd.vehicleId(),
14             VehicleDocument.DocType.valueOf(cmd.docType()),
15             cmd.docNumber(), cmd.issuer(),
16             cmd.issueDate(), cmd.expiryDate(),
17             kernelFile.secureUrl(), // URL du Kernel
18             null, cmd.notes(), null, null
19         );
20         return documentPort.saveVehicleDoc(doc);
21     });
22 }
```

Chapitre 4

Architecture d'Intégration FleetMan — Kernel

4.1 Vue d'Ensemble de l'Architecture Cible

L'architecture cible positionne FleetMan comme un **backend consommateur** du Kernel RT-Comops. FleetMan conserve sa logique métier propre (gestion de flotte, trajets, géorepérage, opérations terrain) tout en déléguant les fonctions transversales au Kernel.

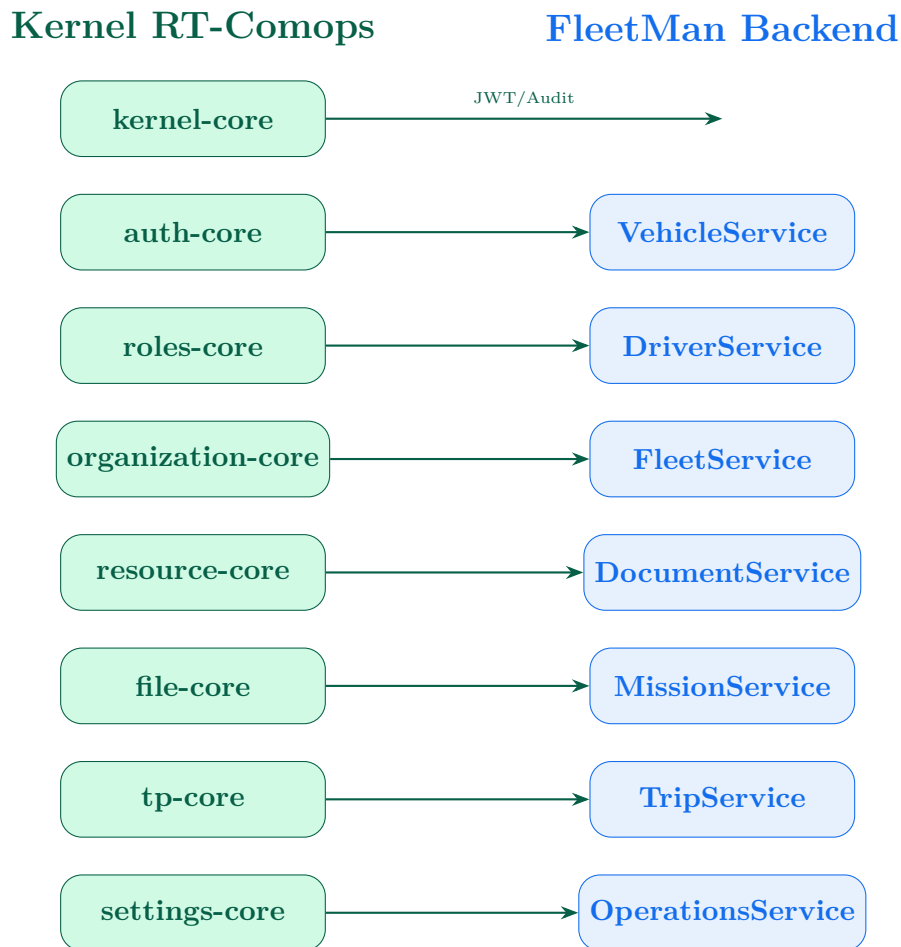


FIG. 4.1 : Architecture d'intégration FleetMan — Kernel RT-Comops

4.2 Couche d'Intégration : ksm-kernel-client

Le Kernel fournit un module client commun `ksm-kernel-client` qui centralise :

- L'appel HTTP vers le Kernel avec injection automatique des headers
- La propagation du contexte (X-Tenant-Id, X-Organization-Id)
- Le relais du bearer token utilisateur
- La gestion des erreurs Kernel (CLIENT_APPLICATION_SERVICE_NOT_ALLOWED, etc.)

Listing 4.1 : Configuration du ksm-kernel-client dans FleetMan

```

1 // pom.xml - Ajout de la dependance
2 <dependency>
3     <groupId>com.rt-comops</groupId>
4     <artifactId>ksm-kernel-client</artifactId>
5     <version>3.0.0</version>
6 </dependency>
7
8 // KernelClientConfig.java
9 @Configuration
10 public class KernelClientConfig {
11
12     @Bean
13     public KernelClient kernelClient(
14         @Value("${application.kernel.url}") String kernelUrl,
15         @Value("${application.kernel.client-id}") String clientId,
16         @Value("${application.kernel.api-key}") String apiKey) {
17
18         return KernelClient.builder()
19             .baseUrl(kernelUrl)
20             .clientId(clientId)
21             .apiKey(apiKey)
22             .build();
23     }
24 }
```

4.3 Gestion des Erreurs Kernel dans FleetMan

FleetMan doit traiter les refus fonctionnels du Kernel comme des signaux de contrat :

Code Erreur Kernel	Traitement FleetMan
CLIENT_APPLICATION_SERVICE_NOT_ALLOWED	OWD critique + alerte admin
TENANT_REQUEST_QUOTA_EXCEEDED	HTTP 429 + message utilisateur
ORGANIZATION_CONTEXT_REQUIRED	HTTP 400 + demande contexte
ORGANIZATION_SERVICE_NOT_SUBSCRIBED	HTTP 403 + message abonnement
ORGANIZATION_SERVICE_QUOTA_EXCEEDED	HTTP 429 + message quota
Refus RBAC	HTTP 403 + message permissions

TAB. 4.1 : Traitement des erreurs Kernel dans FleetMan

Listing 4.2 : Handler d'erreurs Kernel dans GlobalExceptionHandler

```

1 @ExceptionHandler(KernelException.class)
2 public ProblemDetail handleKernelException(KernelException ex) {
3     log.error("Erreur_Kernel_{}:{}", ex.getKernelCode(), ex.
4         getMessage());
5
6     return switch (ex.getKernelCode()) {
7         case "TENANT_REQUEST_QUOTA_EXCEEDED",
8             "ORGANIZATION_SERVICE_QUOTA_EXCEEDED" ->
9             ProblemDetail.forStatusAndDetail(HttpStatus.
10                 TOO_MANY_REQUESTS,
11                 "Quota_de_service_atteint._Veuillez_reessayer_plus_tard
12                 .");
13
14         case "ORGANIZATION_SERVICE_NOT_SUBSCRIBED" ->
15             ProblemDetail.forStatusAndDetail(HttpStatus.FORBIDDEN,
16                 "Votre_organisation_n'est_pas_abonnee_a_ce_service.");
17
18         case "CLIENT_APPLICATION_SERVICE_NOT_ALLOWED" ->
19             ProblemDetail.forStatusAndDetail(HttpStatus.FORBIDDEN,
20                 "Service_non_autorise_pour_cette_application.");
21
22         default ->
23             ProblemDetail.forStatusAndDetail(HttpStatus.BAD_GATEWAY,
24                 "Erreur_du_service_central:_" + ex.getMessage());
25     };
26 }
27 \end{lstlisting}
28
29 %
30 \chapter{Plan d'Implémentation de l'Intégration}
31 %

```

```

29
30 \section{Principes Directeurs}
31
32 L'intégration du Kernel dans FleetMan suit les principes suivants :
33
34 \begin{enumerate}[leftmargin=2cm]
35 \item \textbf{Migration progressive} : FleetMan continue de
36   fonctionner pendant la migration. Les modules locaux sont remplacés
37   un par un.
38 \item \textbf{Anti-Corruption Layer} : des adapters dédiés isolent
39   FleetMan des changements d'API du Kernel.
40 \item \textbf{Données dénormalisées} : FleetMan conserve des copies
41   locales des données Kernel pour les performances (nom du
42   conducteur, immatriculation du véhicule).
43 \item \textbf{Synchronisation par événements} : les changements dans
44   le Kernel sont propagés à FleetMan via Kafka (outbox).
45 \item \textbf{Fallback gracieux} : si le Kernel est indisponible,
46   FleetMan dégrade gracieusement (mode dégradé).
47 \end{enumerate}
48
49 \section{Phase 0 - Préparation (Semaine 1-2)}
50
51 \begin{tcolorbox}[colback=fleetdark!4, colframe=fleetdark, title={\
52   bfseries\color{white} Phase 0 - Enregistrement et Configuration}]
53 \textbf{Objectif :} Enregistrer FleetMan comme ClientApplication dans
54   le Kernel et configurer la connexion.
55
56 \textbf{Tâches :}
57 \begin{itemize}[leftmargin=1.5cm]
58 \item Créer le tenant FleetMan dans le Kernel (via l'admin_Kernel)
59 \item Créer la ClientApplication \texttt{fleet-management-backend}
60 \item Configurer les \texttt{allowedServiceCodes} : ORGANIZATION,
61   RESOURCE, COMMERCIAL, SETTINGS, FILE
62 \item Stocker le secret dans les variables d'environnement
63 \item Ajouter la dépendance \texttt{ksm-kernel-client} dans \texttt{pom.xml}
64 \item Créer \texttt{KernelClientConfig} et \texttt{KernelWebClientAdapter}
65 \item Tester la connexion via l'endpoint \texttt{/kernel/health}
66 \end{itemize}
67
68 \textbf{Livraison :} FleetMan connecté au Kernel, headers propagés
69   correctement.
70 \end{tcolorbox}
71
72 \section{Phase 1 - Intégration auth-core et roles-core (Semaine 3-5)}

```

```

63 \begin{tcolorbox}[colback=fleetblue!5,colframe=fleetblue,title={\
    bfseries\color{white}Phase1-AuthentificationetRBAC
    Centralisés}]
64 \textbf{Objectif:} Remplacer l'authentification locale par l'auth-core
    du Kernel.
65
66 \textbf{Tâches:}
67 \begin{itemize}[leftmargin=1.5cm]
68 \item Configurer la validation JWT RS256 via JWKS du Kernel
69 \item Adapter \texttt{SecurityConfig} pour utiliser \texttt{
    oauth2ResourceServer}
70 \item Migrer \texttt{JwtAuthenticationManager} vers le validateur
    Kernel
71 \item Provisionner les templates de rôles FleetMan dans le Kernel
72 \item Adapter les \texttt{@PreAuthorize} pour utiliser les
    permissions granulaires
73 \item Migrer les utilisateurs existants vers le Kernel (script de
    migration)
74 \item Tester les flows login, refresh, logout
75 \end{itemize}
76
77 \textbf{Livraison:} Authentification 100\% du Kernel, rôles RBAC
    granulaires.
78 \end{tcolorbox}
79
80 \section{Phase2-Intégration organization-core (Semaine 6-7)}
81
82 \begin{tcolorbox}[colback=kernellight,colframe=kernelcolor,title={\
    bfseries\color{white}Phase2-Flottes comme Organisations}]
83 \textbf{Objectif:} Synchroniser les flottes FleetMan avec les
    organisations Kernel.
84
85 \textbf{Tâches:}
86 \begin{itemize}[leftmargin=1.5cm]
87 \item Créer \texttt{KernelOrganizationAdapter} implémentant un port
    sortant
88 \item Modifier \texttt{FleetService.createFleet()} pour créer une
    organisation Kernel
89 \item Ajouter le champ \texttt{kernel\_organization\_id} à \texttt{
    fleet.fleets}
90 \item Créer \texttt{OrganizationEventConsumer} pour synchroniser les
    suspensions
91 \item Propager \texttt{X-Organization-Id} dans tous les appels
    FleetMan
92 \item Migrer les flottes existantes vers des organisations Kernel
93 \end{itemize}
94

```

```

95 \textbf{Livrable :} Flottes synchronisées avec le Kernel, multi-tenancy
    opérationnel.
96 \end{tcolorbox}
97
98 \section{Phase 3 - Intégration resource-core (Semaine 8-10)}
99
100 \begin{tcolorbox}[colback=fleetblue!5, colframe=fleetblue, title={\
    bfseries\color{white}Phase 3 - Véhicules comme Ressources}]
101 \textbf{Objectif :} Remplacer le service véhicule externe par resource-
    core.
102
103 \textbf{Tâches :}
104 \begin{itemize}[leftmargin=1.5cm]
105 \item Créer \texttt{KernelResourceAdapter} remplaçant \texttt{
    VehicleApiAdapter}
106 \item Modifier \texttt{VehicleService} pour déléguer la création au
    Kernel
107 \item Synchroniser les affectations conducteur-véhicule avec resource-
    core
108 \item Intégrer la mise à jour GPS via le port resource-core
109 \item Synchroniser les passages en maintenance avec resource-core
110 \item Migrer les véhicules existants vers des ressources Kernel
111 \end{itemize}
112
113 \textbf{Livrable :} Véhicules gérés par resource-core, GPS synchronisé.
114 \end{tcolorbox}
115
116 \section{Phase 4 - Intégration actor-core et tp-core (Semaine 11-12)}
117
118 \begin{tcolorbox}[colback=kernellight, colframe=kernelcolor, title={\
    bfseries\color{white}Phase 4 - Acteurs et Tiers}]
119 \textbf{Objectif :} Synchroniser conducteurs et clients avec le Kernel.
120
121 \textbf{Tâches :}
122 \begin{itemize}[leftmargin=1.5cm]
123 \item Créer \texttt{KernelActorAdapter} pour la synchronisation des
    conducteurs
124 \item Modifier \texttt{DriverService} pour créer un acteur Kernel à l
    'enregistrement
125 \item Adapter le Module 7 (Clients) pour utiliser tp-core
126 \item Créer \texttt{KernelThirdPartyAdapter}
127 \item Migrer les conducteurs et clients existants
128 \end{itemize}
129
130 \textbf{Livrable :} Conducteurs et clients synchronisés avec le Kernel.
131 \end{tcolorbox}
132

```



```

133 \section{Phase 5 - Intégration file-core et settings-core (Semaine
    13-14)}
134
135 \begin{tcolorbox}[colback=fleetblue!5, colframe=fleetblue, title={\
    bfseries\color{white} Phase 5 - Fichiers et Paramètres}]
136 \textbf{Objectif :} Centraliser les fichiers et paramètres dans le
    Kernel.
137
138 \textbf{Tâches :}
139 \begin{itemize}[leftmargin=1.5cm]
140 \item Créer \texttt{KernelFileAdapter} pour le téléversement des
    documents légaux
141 \item Modifier \texttt{DocumentService} pour utiliser file-core
142 \item Créer \texttt{KernelSettingsAdapter} pour la numérotation des
    missions
143 \item Modifier \texttt{MissionService} pour utiliser settings-core
144 \item Migrer les fichiers existants vers le Kernel
145 \end{itemize}
146
147 \textbf{Livrible :} Documents légaux et numérotation gérés par le
    Kernel.
148 \end{tcolorbox}
149
150 \section{Phase 6 - Intégration common-core et Finalisation (Semaine
    15-16)}
151
152 \begin{tcolorbox}[colback=kernellight, colframe=kernelcolor, title={\
    bfseries\color{white} Phase 6 - Types Partagés et Finalisation}]
153 \textbf{Objectif :} Aligner les types FleetMan sur le common-core et
    finaliser l'intégration.
154
155 \textbf{Tâches :}
156 \begin{itemize}[leftmargin=1.5cm]
157 \item Aligner \texttt{PageResponse<T>} sur \texttt{PageResult<T>} du
    Kernel
158 \item Adopter \texttt{ApiResponse<T>} pour toutes les réponses REST
159 \item Ajouter \texttt{tenantId} sur toutes les entités FleetMan
160 \item Intégrer les adresses polymorphiques pour les flottes et dépôts
161 \item Tests d'intégration end-to-end
162 \item Documentation de l'API mise à jour
163 \end{itemize}
164
165 \textbf{Livrible :} Intégration complète, tests validés, documentation
    à jour.
166 \end{tcolorbox}
167
168 \section{Tableau Récapitulatif du Plan}

```

```

169
170 \begin{longtable}{p{1cm}p{4cm}p{3cm}p{2cm}p{3cm}}
171 \toprule
172 \textbf{Phase}_\&\_\textbf{Modules}_\&\_\textbf{Kernel}_\&\_\textbf{Semaines}_\&\_\textbf{Effort}_\&\_\textbf{Dépendances}_\&\_\textbf{
173 \midrule
174 \endhead
175 0_\&\_\textbf{Enregistrement}_\&\_\textbf{1-2}_\&\_\textbf{S}_\&\_\textbf{Aucune}_\&\_\textbf{
176 1_\&\_\textbf{auth-core}_\&\_\textbf{roles-core}_\&\_\textbf{3-5}_\&\_\textbf{M}_\&\_\textbf{Phase}_\&\_\textbf{0}_\&\_\textbf{
177 2_\&\_\textbf{organization-core}_\&\_\textbf{6-7}_\&\_\textbf{M}_\&\_\textbf{Phase}_\&\_\textbf{1}_\&\_\textbf{
178 3_\&\_\textbf{resource-core}_\&\_\textbf{8-10}_\&\_\textbf{L}_\&\_\textbf{Phase}_\&\_\textbf{2}_\&\_\textbf{
179 4_\&\_\textbf{actor-core}_\&\_\textbf{otp-core}_\&\_\textbf{11-12}_\&\_\textbf{M}_\&\_\textbf{Phase}_\&\_\textbf{2}_\&\_\textbf{
180 5_\&\_\textbf{file-core}_\&\_\textbf{settings-core}_\&\_\textbf{13-14}_\&\_\textbf{M}_\&\_\textbf{Phase}_\&\_\textbf{3}_\&\_\textbf{4}_\&\_\textbf{
181 6_\&\_\textbf{common-core}_\&\_\textbf{finalisation}_\&\_\textbf{15-16}_\&\_\textbf{S}_\&\_\textbf{Phase}_\&\_\textbf{1-5}_\&\_\textbf{
182 \bottomrule
183 \caption{Plan d'implémentation de l'intégration_\&\_\textbf{Kernel}_\&\_\textbf{(16 semaines)}}
184 \end{longtable}
185 %
186 \chapter{Impact_\&\_\textbf{sur}_\&\_\textbf{l'Architecture Existante de FleetMan}}
187 %
188
189 \section{Composants à Remplacer}
190
191 L'intégration_\&\_\textbf{du}_\&\_\textbf{Kernel}_\&\_\textbf{implique}_\&\_\textbf{le}_\&\_\textbf{remplacement}_\&\_\textbf{de}_\&\_\textbf{plusieurs}_\&\_\textbf{
192 composants}
193 existants_\&\_\textbf{de}_\&\_\textbf{FleetMan}_\&\_\textbf{par}_\&\_\textbf{des}_\&\_\textbf{adapters}_\&\_\textbf{vers}_\&\_\textbf{le}_\&\_\textbf{Kernel}_\&\_\textbf{:}
194
195 \begin{longtable}{p{5.5cm}p{5.5cm}p{2cm}}
196 \toprule
197 \textbf{Composant}_\&\_\textbf{FleetMan}_\&\_\textbf{Actuel}_\&\_\textbf{Remplacement}_\&\_\textbf{Kernel}_\&\_\textbf{Phase}_\&\_\textbf{
198 \midrule
199 \endhead
200 \texttt{RemoteAuthAdapter}_\&\_\texttt{KernelAuthAdapter}_\&\_\textbf{1}_\&\_\textbf{
201 \texttt{FakeAuthAdapter}_\&\_\texttt{Supprimé}_\&\_\textbf{1}_\&\_\textbf{
202 \texttt{JwtAuthenticationManager}_\&\_\texttt{Validateur}_\&\_\textbf{JWKS}_\&\_\textbf{Kernel}_\&\_\textbf{1}_\&\_\textbf{
203 \texttt{BearerTokenServerAuthenticationConverter}_\&\_\texttt{Convertisseur}_\&\_\textbf{Kernel}_\&\_\textbf{1}_\&\_\textbf{
204 \texttt{SystemUserInitializer}_\&\_\texttt{Provisionnement}_\&\_\textbf{Kernel}_\&\_\textbf{1}_\&\_\textbf{
205 \texttt{VehicleApiAdapter}_\&\_\texttt{KernelResourceAdapter}_\&\_\textbf{3}_\&\_\textbf{
206 \texttt{VehicleApiClient}_\&\_\texttt{KernelWebClient}_\&\_\textbf{3}_\&\_\textbf{
207 \texttt{AuthApiClient}_\&\_\texttt{KernelAuthClient}_\&\_\textbf{1}_\&\_\textbf{
208 \bottomrule
209 \caption{Composants_\&\_\textbf{FleetMan}_\&\_\textbf{remplacés}_\&\_\textbf{par}_\&\_\textbf{des}_\&\_\textbf{adapters}_\&\_\textbf{Kernel}}
210 \end{longtable}
211
212 \section{Composants_\&\_\textbf{à}_\&\_\textbf{Enrichir}}

```

```

212
213 Ces composants existants sont conservés mais enrichis avec des appels
    Kernel :
214
215 \begin{longtable}{p{5cm}p{8cm}}
216 \toprule
217 \textbf{Composant} & \textbf{Enrichissement} \\
218 \midrule
219 \endhead
220 \texttt{FleetService} & Création d'organisation Kernel à la création de
    flotte \\
221 \texttt{DriverService} & Création d'acteur Kernel à l'enregistrement du
    conducteur \\
222 \texttt{VehicleService} & Enregistrement ressource Kernel +
    synchronisation GPS \\
223 \texttt{MaintenanceService} & Synchronisation état ressource Kernel \\
224 \texttt{DocumentService} & Téléversement fichiers via file-core Kernel
    \\
225 \texttt{MissionService} & Numérotation via settings-core + tiers via tp
    -core \\
226 \texttt{GlobalExceptionHandler} & Gestion des erreurs Kernel \\
227 \texttt{SecurityConfig} & Validation JWT RS256 via JWKS Kernel \\
228 \bottomrule
229 \caption{Composants FleetMan enrichis avec des appels Kernel}
230 \end{longtable}
231
232 \section{Nouveaux Composants à Créer}
233
234 \begin{longtable}{p{6cm}p{7cm}}
235 \toprule
236 \textbf{Nouveau Composant} & \textbf{Rôle} \\
237 \midrule
238 \endhead
239 \texttt{KernelClientConfig} & Configuration du client Kernel \\
240 \texttt{KernelWebClientAdapter} & Injection des headers Kernel \\
241 \texttt{KernelAuthAdapter} & Authentification via auth-core \\
242 \texttt{KernelResourceAdapter} & Gestion ressources via resource-core
    \\
243 \texttt{KernelOrganizationAdapter} & Synchronisation organisations \\
244 \texttt{KernelActorAdapter} & Synchronisation acteurs \\
245 \texttt{KernelThirdPartyAdapter} & Gestion tiers via tp-core \\
246 \texttt{KernelFileAdapter} & Téléversement fichiers via file-core \\
247 \texttt{KernelSettingsAdapter} & Paramètres et numérotation \\
248 \texttt{OrganizationEventConsumer} & Consommateur événements Kafka
    Kernel \\
249 \texttt{ResourceEventConsumer} & Consommateur événements ressources \\
250 \texttt{KernelException} & Exception typée pour les erreurs Kernel

```

```

251 \bottomrule
252 \caption{Nouveaux composants à créer pour l'intégration_Kernel}
253 \end{longtable}
254
255 \section{Impact_sur_la_Base_de_Données}
256
257 L'intégration nécessite l'ajout_de_colonnes_de_référence_vers_le_Kernel
    :
258
259 \begin{lstlisting}[style=yamlStyle, caption={Migration_SQL_pour_les_
    références_Kernel}]
260 --_Migration_020-add-kernel-references.sql
261
262 --_Flottes_:_référence_vers_l'organisation_Kernel
263 ALTER TABLE fleet.fleets
264     ADD COLUMN IF NOT EXISTS kernel_organization_id UUID,
265     ADD COLUMN IF NOT EXISTS kernel_tenant_id UUID;
266
267 -- Vehicules : reference vers la ressource Kernel
268 ALTER TABLE fleet.vehicles
269     ADD COLUMN IF NOT EXISTS kernel_resource_id UUID;
270
271 -- Conducteurs : reference vers l'acteur_Kernel
272 ALTER TABLE fleet.drivers
273     ADD COLUMN IF NOT EXISTS kernel_actor_id UUID;
274
275 --_Missions_:_référence_vers_le_tiers_Kernel
276 ALTER TABLE fleet.missions
277     ADD COLUMN IF NOT EXISTS kernel_third_party_id UUID;
278
279 --_Documents_:_référence_vers_le_fichier_Kernel
280 ALTER TABLE fleet.vehicle_documents
281     ADD COLUMN IF NOT EXISTS kernel_file_id UUID;
282 ALTER TABLE fleet.driver_documents
283     ADD COLUMN IF NOT EXISTS kernel_file_id UUID;
284
285 --_Index_pour_les_jointures
286 CREATE INDEX IF NOT EXISTS idx_fleets_kernel_org
287     ON fleet.fleets (kernel_organization_id);
288 CREATE INDEX IF NOT EXISTS idx_vehicles_kernel_res
289     ON fleet.vehicles (kernel_resource_id);
290 CREATE INDEX IF NOT EXISTS idx_drivers_kernel_actor
291     ON fleet.drivers (kernel_actor_id);

```

Chapitre 5

Patterns d'Intégration Détaillés

5.1 Pattern 1 — Anti-Corruption Layer (ACL)

Chaque module Kernel est encapsulé dans un adapter dédié qui isole FleetMan des changements d'API du Kernel. Ce pattern garantit que si l'API Kernel évolue, seul l'adapter doit être modifié.

Listing 5.1 : Pattern ACL — KernelResourceAdapter

```
1  /**
2   * Anti-Corruption Layer vers resource-core du Kernel.
3   * Isole FleetMan des changements d'API du Kernel.
4   * Implémente ExternalVehiclePort (port sortant existant).
5   */
6   @Component
7   @RequiredArgsConstructor
8   @Slf4j
9   public class KernelResourceAdapter implements ExternalVehiclePort {
10
11       private final KernelWebClientAdapter kernelClient;
12       private final WebClient webClient;
13
14       @Override
15       public Mono<String> registerVehicle(VehicleRequest request,
16                                           String organizationId,
17                                           String token) {
18           // Traduction du modele FleetMan vers le modele Kernel
19           Map<String, Object> kernelRequest = Map.of(
20               "resourceType", "VEHICLE",
21               "name", request.brand() + " " + request.model(),
22               "serialNumber", request.vehicleSerialNumber(),
23               "attributes", Map.of(
24                   "licensePlate", request.licensePlate(),
25                   "color", request.color(),
26                   "year", request.manufacturingYear()
27               )
28           );
```

```

29
30     return kernelClient.withKernelHeaders(
31         webClient.post().uri("/api/v1/resources")
32             .bodyValue(kernelRequest),
33         organizationId, token
34     )
35     .retrieve()
36     .onStatus(status -> status.is4xxClientError(),
37         resp -> resp.bodyToMono(String.class)
38             .flatMap(body -> Mono.error(
39                 new KernelException("RESOURCE_CREATION_FAILED",
40                     body))))
41     .bodyToMono(KernelResourceResponse.class)
42     .map(KernelResourceResponse::id)
43     .doOnSuccess(id -> log.info(
44         "Vehicule_␣enregistre_␣dans_␣Kernel_␣resource-core:␣{}", id));
45
46     // DTO interne de reponse Kernel (isole du domaine FleetMan)
47     private record KernelResourceResponse(String id, String status) {}
48 }

```

5.2 Pattern 2 — Synchronisation par Événements Kafka

Les changements dans le Kernel sont propagés à FleetMan via Kafka. FleetMan consomme les événements du topic `iwm.events.business` et met à jour ses projections locales.

Listing 5.2 : Consommateur d'événements Kernel

```

1  @Component
2  @RequiredArgsConstructor
3  @Slf4j
4  public class KernelEventConsumer {
5
6      private final FleetRepositoryPort fleetPort;
7      private final VehiclePersistencePort vehiclePort;
8      private final DriverPersistencePort driverPort;
9
10     @KafkaListener(
11         topics = "iwm.events.business",
12         groupId = "fleet-management-group",
13         containerFactory = "kernelEventListenerFactory"
14     )
15     public void onKernelEvent(KernelBusinessEvent event) {

```

```

16     log.debug("Evenement_Kernel_recu: type={}, entityId=",
17               event.type(), event.entityId());
18
19     switch (event.type()) {
20     case "ORGANIZATION_SUSPENDED" ->
21         handleOrganizationSuspended(event.entityId());
22     case "RESOURCE_STATUS_CHANGED" ->
23         handleResourceStatusChanged(event.entityId(),
24                                     event.payload());
25     case "ACTOR_DEACTIVATED" ->
26         handleActorDeactivated(event.entityId());
27     default ->
28         log.debug("Evenement_Kernel_ignore: {}", event.type());
29     }
30 }
31
32 private void handleOrganizationSuspended(String orgId) {
33     fleetPort.findByKernelOrganizationId(UUID.fromString(orgId))
34         .flatMap(fleet -> fleetPort.suspend(fleet.id()))
35         .doOnSuccess(f -> log.info(
36             "Flotte_suspendue_suite_a_suspension_organisation_
37             Kernel: {}",
38             orgId))
39         .subscribe();
40 }
41
42 private void handleResourceStatusChanged(String resourceId,
43                                         Map<String, Object>
44                                           payload) {
45     String newStatus = (String) payload.get("status");
46     vehiclePort.findByKernelResourceId(UUID.fromString(resourceId))
47         .flatMap(v -> vehiclePort.updateStatus(v.id(), newStatus))
48         .subscribe();
49 }
50
51 private void handleActorDeactivated(String actorId) {
52     driverPort.findByKernelActorId(UUID.fromString(actorId))
53         .flatMap(d -> driverPort.deactivate(d.userId()))
54         .subscribe();
55 }

```

5.3 Pattern 3 — Données Dénormalisées pour la Performance

FleetMan conserve des copies locales des données Kernel pour éviter des appels HTTP synchrones sur chaque requête. Ces copies sont maintenues à jour via les événements Kafka.

Listing 5.3 : Strategie de denormalisation

```

1 // Dans VehicleService.createIndependentVehicle()
2 // Apres enregistrement dans le Kernel :
3
4 Vehicle vehicle = new Vehicle(
5     null,
6     fleetId,
7     managerId,
8     null,
9     vehicleTypeId,
10    request.licensePlate(),
11    request.vehicleSerialNumber(),
12    request.brand(),
13    request.model(),
14    // ... autres champs locaux ...
15    kernelResourceId // Reference vers le Kernel
16 );
17
18 // La donnee est stockee localement ET referencee dans le Kernel.
19 // Les lectures utilisent la copie locale (performance).
20 // Les ecritures passent par le Kernel (coherence).
```

5.4 Pattern 4 — Fallback Gracieux

Si le Kernel est temporairement indisponible, FleetMan doit dégrader gracieusement sans bloquer les opérations critiques.

Listing 5.4 : Circuit Breaker vers le Kernel

```

1 @Component
2 @RequiredArgsConstructor
3 public class KernelResourceAdapter implements ExternalVehiclePort {
4
5     private final CircuitBreakerFactory circuitBreakerFactory;
6
7     @Override
8     public Mono<String> registerVehicle(VehicleRequest request,
9                                         String orgId, String token) {
```



```

10     CircuitBreaker cb = circuitBreakerFactory
11         .create("kernel-resource");
12
13     return cb.run(
14         // Appel normal vers le Kernel
15         doRegisterVehicle(request, orgId, token),
16         // Fallback : enregistrement local sans Kernel
17         throwable -> {
18             log.warn("Kernel indisponible, mode degrade: {}",
19                 throwable.getMessage());
20             // Retourner un ID temporaire local
21             return Mono.just("LOCAL-" + UUID.randomUUID());
22         }
23     );
24 }
25 }
26 \end{lstlisting}
27
28 %
29 \chapter{Impacts sur la Sécurité et la Conformité}
30 %
31
32 \section{Modèle de Sécurité Cible}
33
34 Avec l'intégration du Kernel, le modèle de sécurité de FleetMan évolue
35 vers un modèle à trois couches :
36
37 \begin{enumerate}[leftmargin=2cm]
38 \item\textbf{Couche Kernel} : authentication, validation JWT RS256
39 ,
40 \item\textbf{Couche FleetMan} : autorisation RBAC via permissions
41 Kernel,
42 \item\textbf{Couche Données} : filtrage systématique par tenantId
43 et managerId dans toutes les requêtes
44 \end{enumerate}
45
46 \section{Cloisonnement Multi-Tenant}
47
48 Chaque entreprise de transport devient un tenant distinct dans le
49 Kernel.
50 Le cloisonnement est garanti par :
51
52 \begin{itemize}[leftmargin=2cm]
53 \item Le tenantId présent dans le JWT et propagé dans toute

```

```

    la
53 \begin{itemize}
54 \item Le filtre \texttt{TenantWebFilter} du Kernel qui injecte le
    contexte
55 \item Les requêtes SQL de FleetMan qui filtrent systématiquement par
56 \texttt{manager\_id} (lié à \texttt{organizationId} Kernel)
57 \item L'impossibilité pour un gestionnaire d'une flotte d'accéder aux
58 données d'une autre flotte (même tenant, organisations
    différentes)
59 \end{itemize}
60
61 \section{Audit et Traçabilité}
62
63 L'intégration du Kernel enrichit considérablement la traçabilité de
    FleetMan:
64
65 \begin{longtable}[p{5cm}p{8cm}]
66 \toprule
67 \textbf{Opération FleetMan} & \textbf{Trace dans le Kernel} \\
68 \midrule
69 \endhead
70 Création d'un véhicule & \texttt{AuditTrail} Kernel + événement outbox
    \\
71 Enregistrement d'un conducteur & \texttt{AuditTrail} Kernel (acteur
    créé) \\
72 Déclaration d'un incident & Événement Kafka \texttt{INCIDENT\_REPORTED}
    \\
73 Téléversement d'un document & \texttt{AuditTrail} Kernel (file-core) \\
74 Connexion d'un utilisateur & \texttt{AuditTrail} Kernel (auth-core) \\
75 Modification d'un rôle & \texttt{AuditTrail} Kernel (roles-core) \\
76 \bottomrule
77 \caption{Traçabilité enrichie par le Kernel}
78 \end{longtable}
79
80 \section{Gestion des Secrets}
81
82 Les secrets de la ClientApplication FleetMan doivent être gérés
83 selon les bonnes pratiques:
84
85 \begin{lstlisting}[style=yamlStyle, caption={Gestion des secrets en
    production}]
86 # Variables d'environnement (jamais en dur dans le code)
87 FLEET_KERNEL_API_KEY=<secret-generé-par-le-kernel>
88 FLEET_KERNEL_CLIENT_ID=fleet-management-backend
89 FLEET_TENANT_ID=<uuid-du-tenant-fleetman>
90
91 # En production : utiliser un gestionnaire de secrets

```

```
92 # (HashiCorp Vault, AWS Secrets Manager, etc.)  
93 # Le secret doit etre pivote regulierement via l'API_Kernel
```

Chapitre 6

Bénéfices et Risques de l'Intégration

6.1 Bénéfices Attendus

6.1.1 Bénéfices Fonctionnels

Domaine	Bénéfice
Authentification	Login multi-contexte, OTP, MFA, réinitialisation mot de passe
Gestion des droits	Permissions granulaires, scopes organisationnels
Multi-tenancy	Isolation complète entre entreprises de transport
Traçabilité	Audit immuable sur toutes les opérations critiques
Fichiers	Stockage centralisé, gouvernance documentaire, expiration
Clients	Cycle de vie complet prospect → client, recherche plein-texte
Numérotation	Séquences documentaires standardisées pour missions/factures
Véhicules	Cycle de vie unifié, localisation GPS centralisée

TAB. 6.1 : Bénéfices fonctionnels de l'intégration Kernel

6.1.2 Bénéfices Techniques

- **Réduction du code** : suppression de 2000 lignes de code d'authentification et de gestion des droits locaux
- **Cohérence** : même modèle de données que les autres modules du projet RT-Comops
- **Évolutivité** : les améliorations du Kernel bénéficient automatiquement à FleetMan

- **Observabilité** : métriques Prometheus unifiées sur tout l'écosystème
- **Sécurité** : JWT RS256 standardisé, rotation des secrets centralisée

6.2 Risques et Mitigations

Risque	Impact	Mitigation
Indisponibilité Kernel	Blocage des opérations	Circuit Breaker + mode dégradé
Latence réseau	Ralentissement des APIs	Cache Redis local + dénormalisation
Rupture d'API Kernel	Régression fonctionnelle	Anti-Corruption Layer (ACL)
Migration des données	Perte de données	Migration progressive + rollback
Quota dépassé	Refus de service	Monitoring + alertes préventives
Secret compromis	Accès non autorisé	Rotation régulière + audit

TAB. 6.2 : Risques et mitigations de l'intégration Kernel

6.3 Métriques de Succès

L'intégration sera considérée réussie si :

- **Fonctionnel** : 100% des flows d'authentification passent par le Kernel
- **Performance** : latence $P95 \leq 200\text{ms}$ sur les endpoints FleetMan (incluant les appels Kernel)
- **Disponibilité** : FleetMan reste opérationnel en mode dégradé si le Kernel est indisponible ≤ 30 secondes
- **Sécurité** : 0 accès cross-tenant détecté en audit
- **Cohérence** : 100% des véhicules FleetMan ont un `kernel_resource_id` valide

Chapitre 7

Spécifications Techniques des Adap- ters

7.1 Structure du Package d'Intégration

Tous les adapters Kernel sont regroupés dans un package dédié :

Listing 7.1 : Structure du package d'intégration Kernel

```
src/main/java/com/yowjob/fleet/  
  infrastructure/  
    adapters/  
      outbound/  
        kernel/                                # Nouveau package  
          auth/  
            KernelAuthAdapter.java            # auth-core  
          resource/  
            KernelResourceAdapter.java        # resource-core  
          organization/  
            KernelOrganizationAdapter.java    # organization-core  
          actor/  
            KernelActorAdapter.java           # actor-core  
          thirdparty/  
            KernelThirdPartyAdapter.java      # tp-core  
          file/  
            KernelFileAdapter.java            # file-core  
          settings/  
            KernelSettingsAdapter.java        # settings-core  
          client/  
            KernelWebClientAdapter.java       # Client commun  
            KernelClientConfig.java          # Configuration  
          dto/  
            KernelResourceResponse.java       # DTOs internes  
            KernelOrganizationResponse.java  
            KernelActorResponse.java  
            KernelFileResponse.java  
          exception/
```

```

        KernelException.java           # Exception typee
inbound/
    messaging/
        KernelEventConsumer.java      # Consommateur Kafka

```

7.2 Interface Commune des Adapters Kernel

Tous les adapters Kernel implémentent une interface commune pour la gestion du contexte :

Listing 7.2 : Interface commune KernelAdapter

```

1  /**
2   * Interface commune pour tous les adapters Kernel.
3   * Garantit la propagation coherente du contexte.
4   */
5  public interface KernelAdapter {
6
7      /**
8       * Construit les headers Kernel pour un appel authentifie.
9       * @param organizationId ID de l'organisation (X-Organization-Id)
10      * @param bearerToken    Token JWT de l'utilisateur
11      */
12      default WebClient.RequestHeadersSpec<?> withContext(
13          WebClient.RequestHeadersSpec<?> spec,
14          String organizationId,
15          String bearerToken) {
16          return spec
17              .header("X-Client-Id", getClientId())
18              .header("X-API-Key", getApiKey())
19              .header("X-Tenant-Id", getTenantId())
20              .header("X-Organization-Id", organizationId)
21              .header("Authorization", "Bearer_" + bearerToken);
22      }
23
24      String getClientId();
25      String getApiKey();
26      String getTenantId();
27  }

```

7.3 Configuration application.yml Enrichie

Listing 7.3 : Configuration complète de l'intégration Kernel

```

application:

```

```
# Configuration Kernel RT-Comops
kernel:
  url: ${KERNEL_URL:https://kernel.rt-comops.com}
  client-id: ${FLEET_KERNEL_CLIENT_ID:fleet-management-backend}
  api-key: ${FLEET_KERNEL_API_KEY}
  tenant-id: ${FLEET_TENANT_ID}
  timeout:
    connect: 5s
    read: 10s
  retry:
    max-attempts: 3
    backoff: 1s
  circuit-breaker:
    failure-rate-threshold: 50
    wait-duration: 30s
  allowed-services:
    - ORGANIZATION
    - RESOURCE
    - COMMERCIAL
    - SETTINGS
    - FILE

# Topics Kafka Kernel
kafka:
  topics:
    kernel-events: iwm.events.business
    operation-events: operation-events-topic
    geofence-alerts: geofence-alerts-topic
```


Chapitre 8

Synthèse et Recommandations

8.1 Synthèse des Modules Kernel Retenus

Module Kernel	Priorité	Valeur pour FleetMan
kernel-core	Critique	Fondation obligatoire : multi-tenancy, JWT, audit, outbox
auth-core	Critique	Remplace l'auth locale, apporte MFA et multi-contexte
roles-core	Critique	RBAC granulaire avec cache Redis
organization-core	Haute	Flotte = Organisation, dépôt = Agence
resource-core	Haute	Véhicules comme ressources, GPS centralisé
common-core	Haute	Types partagés, cohérence structurelle
file-core	Normale	Documents légaux centralisés
tp-core	Normale	Clients des missions comme tiers
settings-core	Normale	Numérotation et paramètres métier
actor-core	Normale	Conducteurs comme acteurs

TAB. 8.1 : Synthèse des modules Kernel retenus pour FleetMan

8.2 Recommandations Finales

1. **Commencer par kernel-core + auth-core** : ces deux modules sont les plus structurants et doivent être intégrés en premier. Toute la sécurité de FleetMan en dépend.
2. **Adopter le pattern ACL systématiquement** : chaque module Kernel doit avoir son propre adapter isolé. Ne jamais appeler directement les APIs Kernel depuis les services métier.

3. **Maintenir les données dénormalisées** : pour les performances, FleetMan doit conserver des copies locales des données Kernel (nom du conducteur, immatriculation, etc.) et les synchroniser via Kafka.
4. **Implémenter le Circuit Breaker dès le début** : le Kernel est un service externe. FleetMan doit être résilient à son indisponibilité.
5. **Tester la migration des données** : avant chaque phase, tester la migration des données existantes vers le Kernel sur un environnement de staging.
6. **Documenter les contrats d'intégration** : maintenir un document vivant des endpoints Kernel utilisés par FleetMan, avec les versions et les changements.
7. **Monitorer les quotas** : configurer des alertes Prometheus lorsque les quotas Kernel atteignent 80% pour éviter les interruptions de service.

8.3 Roadmap Recommandée

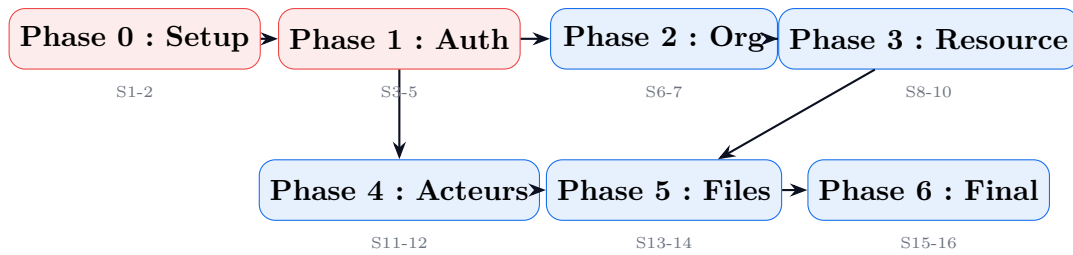


FIG. 8.1 : Roadmap d'intégration Kernel dans FleetMan (16 semaines)

Conclusion

Ce document a présenté une analyse complète des 20 cores du Kernel RT-Comops et identifié les 9 modules directement applicables au projet FleetMan.

L'intégration du Kernel dans FleetMan représente une évolution architecturale majeure qui transforme FleetMan d'une application autonome en un module métier cohérent de l'écosystème RT-Comops. Les bénéfices sont considérables :

- **Sécurité renforcée** : JWT RS256 standardisé, MFA, audit immuable
- **Multi-tenancy réel** : isolation complète entre entreprises de transport
- **Cohérence globale** : même modèle de données que les autres modules
- **Réduction de la dette technique** : suppression du code d'auth local
- **Évolutivité** : bénéfice automatique des améliorations du Kernel

Le plan d'implémentation en 6 phases sur 16 semaines garantit une migration progressive et sécurisée, sans interruption de service. L'adoption du pattern Anti-Corruption Layer protège FleetMan des évolutions futures du Kernel.

Message Clé

FleetMan n'est pas un simple client HTTP du Kernel. Il est une **application consommatrice gouvernée** par le Kernel. La connexion est valide seulement si l'identité machine, le tenant, l'organisation, le service, le quota et le JWT utilisateur sont cohérents. Cette gouvernance est la garantie de la symbiose et de la synchronicité globales du projet RT-Comops.

Annexe A

Glossaire des Termes Kernel

Terme	Définition
Tenant	Unité d'isolation logique de la plateforme (une entreprise)
ClientApplication	Identité machine d'un backend consommateur
allowedServiceCodes	Liste des services autorisés pour une ClientApplication
JWT RS256	Token d'accès signé avec clé RSA 256 bits
JWKS	JSON Web Key Set — document exposant les clés publiques
Outbox	Table transactionnelle pour la publication fiable d'événements
AuditTrail	Piste d'audit immuable (insertion seule)
PartyRef	Référence légère inter-cores (partyType + partyId)
BaseEntity	Classe abstraite avec UUID, tenantId, timestamps
PlatformServiceCode	Énumération des services disponibles dans le Kernel
ACL	Anti-Corruption Layer — adapter isolant un domaine externe
ksm-kernel-client	Module client commun pour les backends consommateurs
X-Client-Id	Header identifiant la ClientApplication
X-API-Key	Header portant le secret de la ClientApplication
X-Tenant-Id	Header identifiant le tenant cible
X-Organization-Id	Header identifiant l'organisation cible

TAB. A.1 : Glossaire des termes Kernel

Annexe B

Checklist d'Intégration

Tâche
Créer le tenant FleetMan dans le Kernel
Créer la ClientApplication <code>fleet-management-backend</code>
Configurer les <code>allowedServiceCodes</code>
Stocker le secret dans les variables d'environnement
Ajouter la dépendance <code>ksm-kernel-client</code>
Créer <code>KernelClientConfig</code>
Configurer la validation JWT RS256 via JWKS
Adapter <code>SecurityConfig</code>
Provisionner les templates de rôles FleetMan
Migrer les utilisateurs existants
Créer <code>KernelOrganizationAdapter</code>
Modifier <code>FleetService</code> pour créer des organisations Kernel
Créer <code>KernelResourceAdapter</code>
Modifier <code>VehicleService</code> pour enregistrer dans <code>resource-core</code>
Créer <code>KernelActorAdapter</code>
Créer <code>KernelThirdPartyAdapter</code>
Créer <code>KernelFileAdapter</code>
Créer <code>KernelSettingsAdapter</code>
Créer <code>KernelEventConsumer</code>
Ajouter la migration SQL <code>020-add-kernel-references.sql</code>
Configurer le Circuit Breaker
Configurer les alertes de quota
Tests d'intégration end-to-end
Documentation API mise à jour

TAB. B.1 : Checklist complète d'intégration Kernel dans FleetMan